Table des matières

Labo 12 OLAP — Le tableau de bord des ventes en ligne 1
Le terrain, en une minute 1
Partie 1 — Cadrer 2
Partie 2 — Reconstituer la série sans trous 4
Partie 3 — Poser les mesures 6
Partie 4 — Contrôler la qualité 7
Partie 5 — Le prix en vigueur, et la marge réelle 10
Partie 6 — Produire le rapport 13
Partie 7 — Mesurer ce que ça coûte 14
Partie 8 — Rendre le chargement rejouable 17
Partie 9 — Ce que le projet vous a montré 19
Nettoyage 19
Et ensuite ? 20

Labo 12 OLAP — Le tableau de bord des ventes en ligne

Laboratoire du Cours 12, voie OLAP · DuckDB + AdventureWorks DW · Formation SQL avancé

C'est le projet de fin de formation. On mène un tableau de bord du cadrage jusqu'au chargement rejouable, en huit parties dont chacune remobilise un module, dans la ligne de commande DuckDB ouverte depuis `Projet_SQL/` sur l'entrepôt. Comptez trois à quatre heures, en une ou deux séances. Rien n'est difficile pris isolément ; ce qui est nouveau, c'est de tout tenir ensemble.

Les deux voies — Ce laboratoire n'a pas de jumeau, et le suffixe `_OLAP` dit sur quel terrain on travaille. Un tableau de bord de ventes est une question analytique, et l'entrepôt est le terrain qui lui convient. Tout ce que vous écrivez ici se transposerait sur PostgreSQL au prix des écarts de syntaxe déjà cartographiés.

Comment travailler ce labo — Ne copiez pas les blocs mécaniquement. À chaque partie, une question vous est posée avant le code : répondez-y d'abord, puis comparez. Le code est la solution, pas l'énoncé.

Le terrain, en une minute

© 2026 iAlexMG. Tous droits réservés.

```sql
USE dbo;

CREATE SCHEMA IF NOT EXISTS RAPPORT;
```

Table	Rôle	Lignes
`dbo.FactInternetSales`	le fait, un article de commande par ligne	60 398
`dbo.DimDate`	l'axe des dates	3 652
`dbo.DimProduct`	le produit, dimension à évolution lente	606
`dbo.DimProductSubcategory` et `dbo.DimProductCategory`	la hiérarchie produit	petites
`dbo.DimCustomer`	le client	18 484
`dbo.DimSalesTerritory`	le territoire	11

On range tout ce qu'on construit dans le schéma `RAPPORT`, pour que le livrable soit identifiable et supprimable d'un bloc.

Partie 1 — Cadrer

Théorie : `Theorie_12_SQL.md`, sections 2 et 3.

Module remobilisé : Cours 01.

La demande est celle-ci, et elle est insuffisante :

Il nous faudrait un tableau de bord des ventes en ligne, avec la tendance et les meilleurs produits.

Avant d'écrire du SQL, répondez à ces cinq questions. Écrivez vos réponses quelque part : elles sont le premier livrable.

1. Que veut dire « ventes » : le montant facturé, le montant hors taxes et hors frais, la marge ?
2. Que veut dire « tendance » : la variation d'un mois sur l'autre, une moyenne mobile, une comparaison à l'an dernier ?
3. Que veut dire « meilleurs produits » : par montant, par quantité, par marge, par nombre de clients distincts ?
4. Sur quelle période, et cette période est-elle complète à ses deux bords ?
5. Qu'est-ce qu'une ligne du rapport représente ?

© 2026 iAlexMG. Tous droits réservés.

1.1 Lire le modèle avant de le croire

```
SELECT COUNT(*)                          AS lignes_de_fait,
       COUNT(DISTINCT SalesOrderNumber)  AS commandes,
       COUNT(DISTINCT CustomerKey)       AS clients,
       COUNT(DISTINCT ProductKey)        AS versions_de_produit_vendues,
       MIN(OrderDate)                    AS premiere_vente,
       MAX(OrderDate)                    AS derniere_vente,
       SUM(SalesAmount)                  AS chiffre_affaires
FROM   dbo.FactInternetSales;
```

Attendu : 60 398 lignes pour 27 659 commandes, 18 484 clients, 158 versions de produit, des ventes du 29 décembre 2010 au 28 janvier 2014, et un chiffre d'affaires de 29 358 677,22 $.

Deux constats à consigner tout de suite.

Une ligne de fait est un article, pas une commande. Le rapport de deux à un entre les deux nombres est le piège numéro un de tout ce laboratoire, et il ne prévient jamais.

La période est tronquée à ses deux bords. Décembre 2010 ne compte que trois jours, janvier 2014 s'arrête au 28. Tout graphique mensuel commencera et finira par une falaise qui n'est pas un fait économique, et le premier lecteur en tirera une conclusion fausse.

1.2 La distribution des articles par commande

```
SELECT articles, COUNT(*) AS commandes
FROM   (SELECT SalesOrderNumber, COUNT(*) AS articles
        FROM dbo.FactInternetSales GROUP BY SalesOrderNumber)
GROUP  BY articles ORDER BY articles;
```

Attendu : de un à huit articles par commande, la majorité en portant un ou deux. Retenez ce tableau : c'est lui qui explique pourquoi sommer une mesure de commande au grain article la gonfle.

1.3 Écrire la note de cadrage

Théorie : `Theorie_12_SQL.md`, sections 2 et 5.

Rédigez maintenant, en dix lignes, la note qui fixe les décisions. Une forme acceptable :

Élément	Décision retenue
Ventes	`SalesAmount`, le montant de la ligne hors taxes et hors frais

© 2026 iAlexMG. Tous droits réservés.

Élément	Décision retenue
Marge	`SalesAmount` moins `TotalProductCost`, au prix en vigueur à la date de la vente
Tendance	moyenne mobile à sept jours sur la série quotidienne, sur un axe complet
Meilleurs produits	par montant, avec la quantité et le nombre de clients en colonnes de contrôle
Période	du 1er janvier 2011 au 31 décembre 2013, les deux mois incomplets étant écartés
Grain du rapport	une ligne par jour pour la tendance, une ligne par produit et par mois pour le classement
Jour sans vente	vaut zéro, et porte une marque qui le distingue d'un zéro observé

La dernière ligne est celle qu'on oublie, et c'est celle qui décidera de la valeur de votre moyenne mobile.

Partie 2 — Reconstituer la série sans trous

Théorie : `Theorie_12_SQL.md`, section 4.

Module remobilisé : Cours 05.

La question avant le code. Combien de jours votre série contient-elle si vous groupez simplement par date, et combien devrait-elle en contenir ?

2.1 Le compte qui alerte

```sql
SELECT COUNT(DISTINCT OrderDate::date) AS jours_actifs,
       DATEDIFF('day', MIN(OrderDate), MAX(OrderDate)) + 1 AS jours_de_la_plage
FROM   dbo.FactInternetSales;
```

Attendu : 1 124 jours actifs pour 1 127 jours de plage. Trois jours sans vente, invisibles tant qu'on ne les cherche pas.

2.2 L'axe, et le piège de la dimension trop large

```sql
-- Le piège : DimDate couvre 2005 à 2014.
SELECT COUNT(*) AS jours_de_la_dimension FROM dbo.DimDate;
```


© 2026 iAlexMG. Tous droits réservés.

Attendu : 3 652. Joindre le fait à toute la dimension donnerait une série qui commence en 2005 avec six ans de zéros. Une dimension de date est faite pour être filtrée, toujours.

```
CREATE OR REPLACE VIEW RAPPORT.V_SERIE_JOUR AS
WITH bornes AS (SELECT DATE '2011-01-01' AS d1, DATE '2013-12-31' AS d2),
axe AS (
    SELECT d.FullDateAlternateKey AS jour
    FROM   dbo.DimDate AS d, bornes AS b
    WHERE  d.FullDateAlternateKey BETWEEN b.d1 AND b.d2
),
brut AS (
    SELECT f.OrderDate::date AS jour,
           SUM(f.SalesAmount)                    AS ventes,
           COUNT(DISTINCT f.SalesOrderNumber)    AS commandes,
           COUNT(*)                              AS articles,
           COUNT(DISTINCT f.CustomerKey)         AS clients
    FROM   dbo.FactInternetSales AS f
    GROUP  BY 1
)
SELECT a.jour,
       COALESCE(b.ventes, 0)    AS ventes,
       COALESCE(b.commandes, 0) AS commandes,
       COALESCE(b.articles, 0)  AS articles,
       COALESCE(b.clients, 0)   AS clients,
       (b.jour IS NULL)         AS jour_reconstitue
FROM   axe AS a
LEFT   JOIN brut AS b ON b.jour = a.jour;

SELECT COUNT(*) AS jours, COUNT_IF(jour_reconstitue) AS jours_sans_vente
FROM   RAPPORT.V_SERIE_JOUR;
```

Attendu : 1 096 jours, dont 3 reconstitués. Notez la colonne `jour_reconstitue` : elle garde la trace de ce qui a été inventé, et elle vous évitera de confondre un zéro observé avec un zéro fabriqué quand quelqu'un vous posera la question dans six mois.

2.3 La preuve que ça compte

```
SELECT AVG(ventes) AS moyenne_axe_complet FROM RAPPORT.V_SERIE_JOUR;
SELECT AVG(ventes) AS moyenne_jours_actifs FROM RAPPORT.V_SERIE_JOUR WHERE NOT jour_reconstitue;
```

Deux nombres différents, deux questions différentes, et une seule des deux répond à « quelle est notre vente quotidienne moyenne ».

© 2026 iAlexMG. Tous droits réservés.

Partie 3 — Poser les mesures

Théorie : `Theorie_12_SQL.md`, section 4.

Module remobilisé : Cours 04.

La question avant le code. Votre moyenne mobile doit-elle porter sur sept lignes ou sur sept jours, et pourquoi la réponse est-elle sans importance ici ?

3.1 Les mesures de tendance

```sql
CREATE OR REPLACE VIEW RAPPORT.V_TENDANCE AS
SELECT jour,
       ventes,
       commandes,
       ventes / NULLIF(commandes, 0) AS panier_moyen,
       AVG(ventes) OVER (ORDER BY jour
                         ROWS BETWEEN 6 PRECEDING AND CURRENT ROW)      AS moyenne_7j,
       SUM(ventes) OVER (PARTITION BY DATE_TRUNC('year', jour)
                         ORDER BY jour
                         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW) AS cumul_annuel,
       ventes - LAG(ventes, 7) OVER (ORDER BY jour)                    AS ecart_semaine_precedente
FROM   RAPPORT.V_SERIE_JOUR;

SELECT * FROM RAPPORT.V_TENDANCE ORDER BY jour LIMIT 20;
```

Trois points de vigilance dans ces quatre lignes.

Le cadre est écrit explicitement partout. Sans lui, le cumul annuel hériterait de `RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW`, ce qui donnerait le même résultat ici parce qu'une ligne égale un jour, et un résultat faux dès qu'une date porterait deux lignes.

Le `ROWS` de la moyenne mobile est légitime parce que l'axe est complet. C'était la réponse à la question posée en tête de partie : sur une série sans trou, le cadre en lignes et le cadre en durée décrivent le même ensemble.

Le `PARTITION BY` du cumul annuel remet le compteur à zéro au 1er janvier, ce que le lecteur métier attend et que personne ne demande explicitement.

3.2 Le classement des produits

```sql
CREATE OR REPLACE VIEW RAPPORT.V_PRODUIT_MOIS AS
SELECT DATE_TRUNC('month', f.OrderDate)::date  AS mois,
       p.ProductAlternateKey                   AS code_produit,
       ANY_VALUE(p.EnglishProductName)         AS produit,
       ANY_VALUE(sc.EnglishProductSubcategoryName) AS sous_categorie,
       ANY_VALUE(c.EnglishProductCategoryName) AS categorie,
```


© 2026 iAlexMG. Tous droits réservés.

```
        SUM(f.SalesAmount)                      AS ventes,
        SUM(f.OrderQuantity)                    AS quantite,
        COUNT(DISTINCT f.CustomerKey)           AS clients,
        COUNT(DISTINCT f.SalesOrderNumber)      AS commandes
FROM    dbo.FactInternetSales AS f
JOIN    dbo.DimProduct AS p  ON p.ProductKey = f.ProductKey
LEFT    JOIN dbo.DimProductSubcategory AS sc ON sc.ProductSubcategoryKey = p.ProductSubcategoryKey
LEFT    JOIN dbo.DimProductCategory    AS c  ON c.ProductCategoryKey     = sc.ProductCategoryKey
WHERE   f.OrderDate >= '2011-01-01' AND f.OrderDate < '2014-01-01'
GROUP   BY 1, 2;

SELECT mois, produit, ventes, quantite, clients,
       RANK()       OVER (PARTITION BY mois ORDER BY ventes DESC) AS rang,
       ROUND(100 * ventes / SUM(ventes) OVER (PARTITION BY mois), 1) AS part_du_mois
FROM   RAPPORT.V_PRODUIT_MOIS
QUALIFY rang <= 5
ORDER  BY mois, rang;
```

Trois décisions se cachent ici, et il faut les défendre.

Le groupement se fait sur `ProductAlternateKey`, la clé métier, et non sur `ProductKey`. Un produit qui a changé de version au cours de la période compterait sinon comme deux produits distincts, et il se classerait deux fois moins bien qu'il ne le mérite.

Les jointures vers la sous-catégorie et la catégorie sont des `LEFT JOIN`. Un produit sans sous-catégorie existe dans cette dimension, et une jointure interne le ferait disparaître du classement sans que rien ne le signale. C'est le piège du `LEFT JOIN` du Cours 01, appliqué à un rapport réel.

Le `RANK` plutôt que `ROW_NUMBER` : deux produits au même montant doivent porter le même rang. Et l'égalité n'a rien d'improbable : dès janvier 2011, les Road-150 Red en taille 62 et en taille 44 font chacun 78 721,94 $ et partagent le rang 2, et le suivant prend le rang 4. `ROW_NUMBER` en aurait déclaré un deuxième et l'autre troisième, au hasard.

Un raccourci et une part, enfin. `ANY_VALUE` rend l'une des valeurs du groupe, sans dire laquelle : c'est acceptable pour un libellé, alors que grouper aussi sur le nom couperait de nouveau en deux un produit dont le libellé a changé d'une version à l'autre. La part du mois, `ventes / SUM(ventes) OVER (PARTITION BY mois)`, est la part du total du Cours 04.

Portabilité — Snowflake a pour cette part un raccourci, `RATIO_TO_REPORT(ventes) OVER (PARTITION BY mois)`, que DuckDB n'a pas : `Aggregate Function with name ratio_to_report does not exist!`. La forme longue, elle, passe partout.

Partie 4 — Contrôler la qualité

Théorie : `Theorie_12_SQL.md`, section 4.

© 2026 iAlexMG. Tous droits réservés.

Module remobilisé : Cours 07.

La question avant le code. Un contrôle qui ne trouve rien est-il un contrôle inutile ?

4.1 Les anti-jointures de référence

```
-- Des lignes de fait sans produit ?
SELECT COUNT(*) AS faits_sans_produit
FROM   dbo.FactInternetSales AS f
WHERE  NOT EXISTS (SELECT 1 FROM dbo.DimProduct AS p WHERE p.ProductKey = f.ProductKey);

-- Sans client ?
SELECT COUNT(*) AS faits_sans_client
FROM   dbo.FactInternetSales AS f
WHERE  NOT EXISTS (SELECT 1 FROM dbo.DimCustomer AS c WHERE c.CustomerKey = f.CustomerKey);

-- Sans date ?
SELECT COUNT(*) AS faits_sans_date
FROM   dbo.FactInternetSales AS f
WHERE  NOT EXISTS (SELECT 1 FROM dbo.DimDate AS d WHERE d.DateKey = f.OrderDateKey);

-- Sans territoire ?
SELECT COUNT(*) AS faits_sans_territoire
FROM   dbo.FactInternetSales AS f
WHERE  NOT EXISTS (SELECT 1 FROM dbo.DimSalesTerritory AS t
                   WHERE t.SalesTerritoryKey = f.SalesTerritoryKey);
```

Attendu : zéro partout. C'est la réponse à la question posée en tête de partie : un contrôle qui ne trouve rien vous apprend que le chiffre suivant est solide, et il vaut d'être conservé dans le livrable. Le jour où il trouvera quelque chose, vous saurez que la situation est nouvelle.

4.2 Les contrôles qui, eux, trouvent quelque chose

```
-- Des lignes de dimension jamais employées.
SELECT COUNT(*) AS produits_jamais_vendus
FROM   dbo.DimProduct AS p
WHERE  NOT EXISTS (SELECT 1 FROM dbo.FactInternetSales AS f WHERE f.ProductKey = p.ProductKey);

SELECT COUNT(*) AS territoires_sans_vente_en_ligne
FROM   dbo.DimSalesTerritory AS t
WHERE  NOT EXISTS (SELECT 1 FROM dbo.FactInternetSales AS f
                   WHERE f.SalesTerritoryKey = t.SalesTerritoryKey);
```


© 2026 iAlexMG. Tous droits réservés.

```sql
-- Les intervalles de validité de la dimension produit.
SELECT COUNT(*) AS dates_de_validite_incoherentes
FROM   dbo.DimProduct
WHERE  EndDate IS NOT NULL AND EndDate < StartDate;
```

Attendu : 448 lignes de produit jamais vendues sur 606, un territoire sans vente en ligne, et 200 intervalles incohérents.

Les deux premiers ne sont pas des anomalies : un catalogue contient des produits qui ne se vendent pas en ligne, et le territoire restant est le siège social. Il faut savoir les regarder et conclure qu'il n'y a rien à faire, ce qui est un exercice plus difficile qu'il n'y paraît.

Le troisième est une vraie anomalie, et c'est celle du Cours 09. Elle décide de la façon dont vous allez écrire la partie 5.

4.3 Les contrôles de grain et de cohérence

Théorie : `Theorie_02_SQL.md`, section 4.

```sql
-- Une clé de ligne de commande est-elle unique ?
SELECT COUNT(*) AS doublons_de_ligne
FROM   (SELECT SalesOrderNumber, SalesOrderLineNumber, COUNT(*) AS n
        FROM dbo.FactInternetSales GROUP BY 1, 2 HAVING COUNT(*) > 1);

-- Le montant de ligne est-il cohérent avec quantité et prix unitaire ?
SELECT COUNT(*) AS lignes_incoherentes
FROM   dbo.FactInternetSales
WHERE  ABS(SalesAmount - OrderQuantity * UnitPrice) > 0.01;

-- Des montants négatifs ou nuls ?
SELECT COUNT_IF(SalesAmount <= 0) AS montants_non_positifs,
       COUNT_IF(OrderQuantity <= 0) AS quantites_non_positives
FROM   dbo.FactInternetSales;
```

Attendu : quatre zéros. Aucun doublon de clé de ligne, aucune incohérence entre le montant et ses composantes, aucun montant ni quantité non positifs. Notez-les dans votre rapport de contrôle exactement comme s'ils avaient trouvé quelque chose : un contrôle qui ne trouve rien n'est pas un contrôle inutile, c'est un contrôle qui a réussi, et c'est la seule preuve que vous aurez à donner si quelqu'un conteste un chiffre dans six mois.

Le deuxième contrôle est celui qu'on n'écrit jamais et qui rapporte le plus : il vérifie qu'une mesure pré-calculée est cohérente avec ses composantes. Une divergence signalerait que la remise n'est pas prise en compte comme vous le croyez, ou qu'une ligne a été corrigée d'un côté et pas de l'autre. Que les 60 398 lignes passent ici est une information, et elle vous autorise à employer `SalesAmount` sans le recalculer dans tout le reste du laboratoire.

4.4 Ranger les contrôles dans le livrable

Théorie : `Theorie_02_SQL.md`, section 3.

© 2026 iAlexMG. Tous droits réservés.

```sql
CREATE OR REPLACE VIEW RAPPORT.V_CONTROLES AS
SELECT 'faits sans produit' AS controle,
       (SELECT COUNT(*) FROM dbo.FactInternetSales AS f
        WHERE NOT EXISTS (SELECT 1 FROM dbo.DimProduct AS p WHERE p.ProductKey = f.ProductKey)) AS valeur,
       0 AS seuil_max
UNION ALL
SELECT 'doublons de ligne',
       (SELECT COUNT(*) FROM (SELECT SalesOrderNumber, SalesOrderLineNumber
                              FROM dbo.FactInternetSales
                              GROUP BY 1, 2 HAVING COUNT(*) > 1)), 0
UNION ALL
SELECT 'lignes incoherentes',
       (SELECT COUNT(*) FROM dbo.FactInternetSales
        WHERE ABS(SalesAmount - OrderQuantity * UnitPrice) > 0.01), 0
UNION ALL
SELECT 'intervalles de validite incoherents',
       (SELECT COUNT(*) FROM dbo.DimProduct
        WHERE EndDate IS NOT NULL AND EndDate < StartDate), 0;

SELECT controle, valeur, seuil_max,
       IF(valeur <= seuil_max, 'OK', 'A REGARDER') AS verdict
FROM   RAPPORT.V_CONTROLES;
```

Vous venez de transformer quatre requêtes jetables en un tableau de bord de qualité. Le quatrième contrôle sortira en alerte, et c'est voulu : il documente un défaut connu, dont la partie 5 explique qu'on a choisi de vivre avec.

Partie 5 — Le prix en vigueur, et la marge réelle

Théorie : `Theorie_12_SQL.md`, section 4.

Module remobilisé : Cours 09.

La question avant le code. Le fait porte déjà `ProductStandardCost` et `TotalProductCost`. Pourquoi ne pas s'en contenter ?

5.1 Pourquoi l'as-of join, malgré les colonnes déjà là

Le fait porte le coût du produit tel qu'il était au moment du chargement de cette ligne, ce qui est généralement le bon chiffre. Mais rien ne le garantit, et vous avez maintenant les moyens de le vérifier plutôt que de le croire. C'est exactement ce qu'on demande à un ingénieur analytique.

```sql
CREATE OR REPLACE VIEW RAPPORT.V_LIGNE_ENRICHIE AS
SELECT f.SalesOrderNumber,
```


© 2026 iAlexMG. Tous droits réservés.

```
        f.SalesOrderLineNumber,
        f.OrderDate::date            AS jour,
        f.CustomerKey,
        ref.ProductAlternateKey      AS code_produit,
        f.ProductKey                 AS cle_portee_par_le_fait,
        v.ProductKey                 AS cle_selon_les_dates,
        f.OrderQuantity,
        f.SalesAmount,
        f.TotalProductCost,
        f.SalesAmount - f.TotalProductCost AS marge_du_fait,
        v.ListPrice                  AS prix_de_liste_du_jour,
        v.StandardCost               AS cout_standard_du_jour,
        f.SalesAmount - v.StandardCost * f.OrderQuantity AS marge_recalculee
FROM    dbo.FactInternetSales AS f
JOIN    dbo.DimProduct AS ref ON ref.ProductKey = f.ProductKey
LEFT    JOIN LATERAL (
          SELECT p.ProductKey, p.ListPrice, p.StandardCost
          FROM   dbo.DimProduct AS p
          WHERE  p.ProductAlternateKey = ref.ProductAlternateKey
            AND  p.StartDate <= f.OrderDate
          ORDER  BY p.StartDate DESC
          LIMIT  1
        ) AS v ON TRUE
WHERE   f.OrderDate >= '2011-01-01' AND f.OrderDate < '2014-01-01';
```

L'écriture n'emploie que `StartDate`, jamais `EndDate`, et c'est ce qui la rend insensible aux 200 intervalles cassés relevés en partie 4. Le `LEFT` est ce qui rend visibles les lignes sans version valide.

5.2 Le verdict

```
SELECT CASE WHEN cle_selon_les_dates IS NULL THEN 'aucune version valide'
            WHEN cle_selon_les_dates = cle_portee_par_le_fait THEN 'accord'
            ELSE 'desaccord' END AS verdict,
       COUNT(*)          AS lignes,
       SUM(SalesAmount)  AS montant
FROM   RAPPORT.V_LIGNE_ENRICHIE
GROUP  BY 1 ORDER BY 2 DESC;
```

Attendu : trois populations, et leur taille relative est le vrai enseignement de la partie.

© 2026 iAlexMG. Tous droits réservés.

Verdict	Lignes	Montant
accord	33 119	16 529 578,02 $
aucune version valide	18 396	8 606 760,92 $
desaccord	6 899	4 133 222,53 $

Regardez la deuxième ligne avant les autres. Près d'un tiers des lignes n'ont aucune version de produit valide à la date de la vente, ce qui n'est pas un défaut de votre requête mais l'état réel de la dimension, dont les intervalles de validité sont incohérents (partie 4.2). Et 6 899 lignes sont en désaccord franc : la clé que le fait porte n'est pas celle que les dates désignent.

Vous obtenez donc trois populations, et il faut décider quoi en faire. C'est une décision de cadrage, pas une décision technique, et elle appartient à votre note. Trois options se défendent.

Option	Ce qu'on fait	Quand elle convient
Faire confiance au fait	employer `TotalProductCost`, et documenter l'écart	la marge n'est pas la mesure principale du rapport
Recalculer	employer le coût retrouvé par as-of join, et écarter les lignes sans version	la marge est la mesure principale, et l'écart est explicable
Publier les deux	deux colonnes, et un contrôle qui suit leur écart dans le temps	on ne sait pas encore, et on veut le savoir

La troisième est souvent la bonne pendant les premières semaines d'un rapport. Elle a un défaut qu'il faut assumer : deux colonnes de marge dans un tableau de bord invitent le lecteur à choisir celle qui l'arrange.

5.3 L'écart, mesuré

```sql
SELECT ROUND(SUM(marge_du_fait), 2)                       AS marge_selon_le_fait,
       ROUND(SUM(marge_recalculee), 2)                    AS marge_recalculee,
       ROUND(SUM(marge_du_fait) - SUM(marge_recalculee), 2) AS ecart,
       ROUND(100 * (SUM(marge_du_fait) - SUM(marge_recalculee))
             / NULLIF(SUM(marge_du_fait), 0), 2)          AS ecart_pct
FROM   RAPPORT.V_LIGNE_ENRICHIE
WHERE  cle_selon_les_dates IS NOT NULL;
```

Attendu : 8 558 623,91 $ de marge selon le fait, 8 870 943,24 $ recalculée, soit un écart de -312 319,33 $, ou -3,65 %. Le signe compte : la marge recalculée est la plus élevée des deux, donc le fait sous-estime la marge plutôt que de la gonfler, ce qui est la direction la moins gênante des deux mais ne rend pas l'écart acceptable.

© 2026 iAlexMG. Tous droits réservés.

Un écart en pourcentage est ce qui permet d'arbitrer. Sous un dixième de point, la question ne mérite pas de retarder la livraison ; au delà de quelques points, elle doit remonter avant publication. Trois points et demi tombent franchement dans le second cas, et c'est la conclusion à écrire dans votre note : ce rapport ne se publie pas sans que quelqu'un ait tranché laquelle des deux marges fait foi. Remarquez que ce chiffre ne porte que sur les 40 018 lignes qui ont une version valide ; les 18 396 autres sont hors du calcul, et une note honnête le dit aussi.

Partie 6 — Produire le rapport

Théorie : `Theorie_12_SQL.md`, sections 4 et 5.

Module remobilisé : Cours 06.

La question avant le code. Pourquoi produire le rapport au format long plutôt qu'en tableau croisé, alors que le lecteur veut un tableau croisé ?

6.1 Le rapport avec ses sous-totaux

```sql
CREATE OR REPLACE VIEW RAPPORT.V_SYNTHESE AS
SELECT COALESCE(c.EnglishProductCategoryName, '(toutes categories)') AS categorie,
       COALESCE(sc.EnglishProductSubcategoryName, '(toutes sous-categories)') AS sous_categorie,
       COALESCE(t.SalesTerritoryCountry, '(tous pays)')              AS pays,
       GROUPING(c.EnglishProductCategoryName)     AS total_categorie,
       GROUPING(sc.EnglishProductSubcategoryName) AS total_sous_categorie,
       GROUPING(t.SalesTerritoryCountry)          AS total_pays,
       SUM(f.SalesAmount)                         AS ventes,
       COUNT(DISTINCT f.SalesOrderNumber)         AS commandes,
       SUM(f.OrderQuantity)                       AS quantite
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimProduct AS p ON p.ProductKey = f.ProductKey
LEFT   JOIN dbo.DimProductSubcategory AS sc ON sc.ProductSubcategoryKey = p.ProductSubcategoryKey
LEFT   JOIN dbo.DimProductCategory    AS c  ON c.ProductCategoryKey     = sc.ProductCategoryKey
LEFT   JOIN dbo.DimSalesTerritory     AS t  ON t.SalesTerritoryKey      = f.SalesTerritoryKey
WHERE  f.OrderDate >= '2011-01-01' AND f.OrderDate < '2014-01-01'
GROUP  BY ROLLUP (c.EnglishProductCategoryName,
                  sc.EnglishProductSubcategoryName,
                  t.SalesTerritoryCountry);

SELECT * FROM RAPPORT.V_SYNTHESE ORDER BY categorie, sous_categorie, pays LIMIT 40;
```

Le `ROLLUP` est une cascade, et l'ordre des trois colonnes décide de ce qu'on obtient : le détail par pays, le sous-total par sous-catégorie, le sous-total par catégorie, puis le total général. Il ne donne pas les ventes par pays toutes catégories confondues, et c'est le `CUBE` du Cours 06 qui les ajouterait, au prix de niveaux dont personne n'a besoin ici.

© 2026 iAlexMG. Tous droits réservés.

Les colonnes issues de `GROUPING` sont indispensables et souvent oubliées. Sans elles, rien ne distingue une ligne de sous-total d'une ligne dont la catégorie est réellement inconnue, et le `COALESCE` qui met une étiquette lisible aggrave la confusion au lieu de la lever.

6.2 Le format long, et pourquoi

```
CREATE OR REPLACE VIEW RAPPORT.V_MESURES_LONGUES AS
SELECT jour, 'ventes'       AS mesure, ventes       AS valeur FROM RAPPORT.V_SERIE_JOUR
UNION ALL
SELECT jour, 'commandes',   commandes   FROM RAPPORT.V_SERIE_JOUR
UNION ALL
SELECT jour, 'articles',    articles    FROM RAPPORT.V_SERIE_JOUR
UNION ALL
SELECT jour, 'clients',     clients     FROM RAPPORT.V_SERIE_JOUR;

SELECT * FROM RAPPORT.V_MESURES_LONGUES ORDER BY jour, mesure LIMIT 20;
```

Voilà la réponse à la question posée en tête de partie. Le lecteur veut un tableau croisé, et il le veut dans son outil de restitution, pas dans votre SQL. Un rapport livré au format long accepte une mesure de plus sans changer de structure, alors qu'un tableau croisé figé dans une vue oblige à réécrire la vue à chaque ajout. La transposition est le travail de la dernière étape, celle qui affiche, et c'est là qu'elle coûte le moins cher.

Si vous voulez tout de même la voir :

```
SELECT * FROM RAPPORT.V_MESURES_LONGUES
PIVOT (SUM(valeur) FOR mesure IN ('ventes', 'commandes', 'articles', 'clients'))
ORDER  BY jour LIMIT 20;
```

Les colonnes produites portent le nom de la valeur pivotée, `ventes`, `commandes`, et toutes sortent au même type décimal, parce que l'`UNION ALL` du format long a donné un seul type à la colonne `valeur` : les comptes s'affichent `2.0000`. C'est une raison de plus de laisser cette transposition à l'outil de restitution, qui connaît le type de chaque mesure.

Portabilité — Snowflake accepte la même écriture du `PIVOT`, mais, d'après sa documentation, garde les guillemets dans le nom des colonnes produites (`'ventes'`), qu'une liste d'alias doit renommer.

Partie 7 — Mesurer ce que ça coûte

Théorie : `Theorie_12_SQL.md`, section 4.

Module remobilisé : Cours 10.

La question avant le code. Laquelle de vos vues coûte le plus cher, et sur quoi pariez-vous avant de regarder ?

© 2026 iAlexMG. Tous droits réservés.

7.1 Chronométrer

```
EXPLAIN ANALYZE SELECT * FROM RAPPORT.V_TENDANCE;
EXPLAIN ANALYZE SELECT * FROM RAPPORT.V_PRODUIT_MOIS;
EXPLAIN ANALYZE SELECT * FROM RAPPORT.V_LIGNE_ENRICHIE;
EXPLAIN ANALYZE SELECT * FROM RAPPORT.V_SYNTHESE;
```

`EXPLAIN ANALYZE` exécute la requête en entier, n'en affiche pas le résultat, et rend son profil : le temps total en tête, `Total Time`, puis chaque opérateur, avec le nombre de lignes qu'il a fait passer et le temps qu'il a pris. Un `SELECT COUNT(*)` ne mesurerait pas la vue : le moteur n'y calcule que ce qu'il faut pour compter, et laisse tomber les colonnes que personne ne lit.

Attendu, sur notre poste : une vingtaine de millisecondes pour `V_TENDANCE`, une trentaine pour `V_LIGNE_ENRICHIE`, près de quarante pour `V_PRODUIT_MOIS`, et environ soixante-dix pour `V_SYNTHESE`, la plus lente. Vos temps dépendront de votre poste ; c'est leur ordre qui compte. Dans le profil de `V_SYNTHESE`, un opérateur porte l'essentiel du temps : le `HASH_GROUP_BY` du `ROLLUP`, avec son `COUNT(DISTINCT …)`, qui ramène 58 414 lignes aux 123 du rapport.

7.2 L'as-of join, et son coût

On aurait parié sur `V_LIGNE_ENRICHIE`, dont la sous-requête latérale a l'air de s'exécuter une fois par ligne de fait. Son profil dit autre chose : un `LEFT_DELIM_JOIN`, la décorrélation du Cours 07, et aucune boucle. Écrivez quand même la variante en `QUALIFY`, et comparez.

```
CREATE OR REPLACE VIEW RAPPORT.V_LIGNE_ENRICHIE_BIS AS
SELECT f.SalesOrderNumber, f.SalesOrderLineNumber, f.OrderDate::date AS jour,
       ref.ProductAlternateKey AS code_produit,
       f.ProductKey AS cle_portee_par_le_fait,
       v.ProductKey AS cle_selon_les_dates,
       f.SalesAmount,
       f.SalesAmount - v.StandardCost * f.OrderQuantity AS marge_recalculee
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimProduct AS ref ON ref.ProductKey = f.ProductKey
LEFT   JOIN dbo.DimProduct AS v
       ON v.ProductAlternateKey = ref.ProductAlternateKey
      AND v.StartDate <= f.OrderDate
WHERE  f.OrderDate >= '2011-01-01' AND f.OrderDate < '2014-01-01'
QUALIFY ROW_NUMBER() OVER (PARTITION BY f.SalesOrderNumber, f.SalesOrderLineNumber
                           ORDER BY v.StartDate DESC) = 1;

-- Les deux doivent donner le même verdict.
SELECT COUNT(*), COUNT(cle_selon_les_dates) FROM RAPPORT.V_LIGNE_ENRICHIE;
SELECT COUNT(*), COUNT(cle_selon_les_dates) FROM RAPPORT.V_LIGNE_ENRICHIE_BIS;

EXPLAIN ANALYZE SELECT * FROM RAPPORT.V_LIGNE_ENRICHIE_BIS;
```


© 2026 iAlexMG. Tous droits réservés.

Vérifiez d'abord l'égalité des résultats, 58 414 lignes et 40 018 versions trouvées des deux côtés, puis seulement les temps. Une optimisation qui change le résultat n'est pas une optimisation. Ici, la variante n'optimise rien : à chacun de nos essais, elle a pris un peu plus de temps que l'originale, de quelques millisecondes. Le moteur avait déjà fait le travail, et c'est la mesure qui l'a montré, pas l'intuition.

7.3 La sargabilité, une dernière fois

```
SELECT SUM(SalesAmount) FROM dbo.FactInternetSales WHERE YEAR(OrderDate) = 2013;
SELECT SUM(SalesAmount) FROM dbo.FactInternetSales
WHERE  OrderDate >= '2013-01-01' AND OrderDate < '2014-01-01';
```

Même résultat, 16 351 550,34 $, et même durée, quelques millisecondes. Sur l'entrepôt de la formation, la différence ne se voit pas. DuckDB range une table par groupes d'au plus 122 880 lignes, et garde le minimum et le maximum de chaque colonne dans chaque groupe ; `PRAGMA_STORAGE_INFO` les montre. Les 60 398 ventes tiennent dans un seul groupe, qu'il faut lire de toute façon. Sur une grosse table, c'est autre chose. Fabriquez-en une, temporaire, de vingt millions de lignes rangées par date :

```
.timer on

CREATE TEMP TABLE ventes_massives AS
SELECT n AS id,
       TIMESTAMP '2010-01-01' + TO_SECONDS(n * 6) AS moment,
       (n % 1000) / 10.0 AS montant
FROM   RANGE(20000000) AS r(n);

SELECT SUM(montant) FROM ventes_massives WHERE YEAR(moment) = 2013;
SELECT SUM(montant) FROM ventes_massives
WHERE  moment >= '2013-01-01' AND moment < '2014-01-01';

DROP TABLE ventes_massives;

.timer off
```

Attendu : la même somme des deux côtés, 210 681 120, en un peu plus de vingt millisecondes pour la première forme et en une ou deux pour la seconde, sur notre poste. La table compte 163 groupes. La plage compare la colonne elle-même à deux bornes : DuckDB écarte, sans les lire, les groupes dont le minimum et le maximum tombent hors de 2013. La fonction l'en empêche, puisqu'il faut calculer `YEAR` sur chaque ligne pour savoir. Une table temporaire vit dans la session, et le fichier de l'entrepôt n'a pas grossi. Vérifiez que vos vues emploient partout la seconde forme.

7.4 Matérialiser ce qui mérite de l'être

```
CREATE OR REPLACE TABLE RAPPORT.T_LIGNE_ENRICHIE AS
SELECT * FROM RAPPORT.V_LIGNE_ENRICHIE;

SELECT COUNT(*) FROM RAPPORT.T_LIGNE_ENRICHIE;
```


© 2026 iAlexMG. Tous droits réservés.

Une vue se recalcule à chaque lecture. Quand la même couche intermédiaire sert cinq rapports et qu'elle coûte cher, on la matérialise, et on accepte en échange qu'elle doive être rafraîchie. C'est un compromis, pas une amélioration gratuite, et c'est le sujet de la partie 8.

Écrivez maintenant votre note de performance : ce que coûtait chaque vue, ce que vous avez changé, ce que ça a donné, et ce que vous avez décidé de ne pas changer.

Partie 8 — Rendre le chargement rejouable

Théorie : `Theorie_12_SQL.md`, section 4.

Module remobilisé : Cours 11.

La question avant le code. Que voit un lecteur du rapport pendant que vous rafraîchissez `T_LIGNE_ENRICHIE` ?

8.1 Le rafraîchissement naïf, et sa fenêtre

```
-- Ne faites pas ça.
DELETE FROM RAPPORT.T_LIGNE_ENRICHIE;
INSERT INTO RAPPORT.T_LIGNE_ENRICHIE SELECT * FROM RAPPORT.V_LIGNE_ENRICHIE;
```

Entre les deux commandes, la table est vide. Tout rapport lu à cet instant affiche zéro, sans erreur et sans avertissement.

8.2 La version atomique

```
BEGIN;
DELETE FROM RAPPORT.T_LIGNE_ENRICHIE;
INSERT INTO RAPPORT.T_LIGNE_ENRICHIE SELECT * FROM RAPPORT.V_LIGNE_ENRICHIE;
COMMIT;
```

Aucun lecteur ne voit l'état intermédiaire. Le chargement est aussi idempotent : lancé deux fois, il laisse le même contenu.

8.3 La version par tranche, celle qui passe à l'échelle

Reconstruire toute la table chaque nuit est acceptable sur 60 000 lignes et absurde sur 200 millions. Le motif qui passe à l'échelle recharge la tranche concernée. La journée à recharger est le seul paramètre : on la range dans une variable de session, que `SET VARIABLE` pose et que `GETVARIABLE` relit.

```
SET VARIABLE jour_a_charger = DATE '2013-06-17';

BEGIN;
DELETE FROM RAPPORT.T_LIGNE_ENRICHIE WHERE jour = GETVARIABLE('jour_a_charger');
INSERT INTO RAPPORT.T_LIGNE_ENRICHIE
```


© 2026 iAlexMG. Tous droits réservés.

```
SELECT * FROM RAPPORT.V_LIGNE_ENRICHIE WHERE jour = GETVARIABLE('jour_a_charger');
COMMIT;

SELECT COUNT(*) FROM RAPPORT.T_LIGNE_ENRICHIE WHERE jour = GETVARIABLE('jour_a_charger');
```

Relancez le bloc, du `BEGIN` au `COMMIT`, puis recomptez : même résultat. Le `DELETE` et l'`INSERT` portent sur exactement la même tranche. C'est ce qui rend le chargement idempotent, et c'est la seule chose à vérifier quand on relit ce genre de code. Dans un vrai pipeline, ces quatre lignes vivent dans un script que l'orchestrateur lance chaque nuit avec la date du jour.

Portabilité — DuckDB n'a pas de procédure stockée. Snowflake et PostgreSQL enveloppent ce même motif dans une procédure, `CREATE PROCEDURE … AS $$ … $$`, qui reçoit la journée en paramètre et s'appelle par `CALL`. La règle d'idempotence ne change pas : même tranche au `DELETE` et à l'`INSERT`, dans une seule transaction.

8.4 Le journal, pour savoir au matin

Théorie : `Theorie_02_SQL.md`, section 6.

```
CREATE OR REPLACE TABLE RAPPORT.JOURNAL_CHARGEMENT (
    etape       VARCHAR,
    debut       TIMESTAMP,
    fin         TIMESTAMP,
    lignes      BIGINT,
    statut      VARCHAR
);

INSERT INTO RAPPORT.JOURNAL_CHARGEMENT
SELECT 'ligne_enrichie', CURRENT_TIMESTAMP, CURRENT_TIMESTAMP,
       (SELECT COUNT(*) FROM RAPPORT.T_LIGNE_ENRICHIE), 'OK';

SELECT * FROM RAPPORT.JOURNAL_CHARGEMENT ORDER BY fin DESC;
```

C'est cette table que l'équipe regarde le matin, pas des comptes de lignes calculés à la main. Une version plus complète enregistre une ligne par étape, avec son statut, et une dernière ligne qui marque l'achèvement global.

8.5 Le filet de sécurité, et ses limites

DuckDB ne garde aucune version passée d'une table. Son filet est plus rustique : l'entrepôt tient dans un seul fichier, `adventureworksdw.duckdb`, et une copie de ce fichier, faite avant un chargement risqué, ligne de commande fermée, suffit à revenir en arrière. Ce filet ne remplace pas la transaction de la sous-partie 8.2 : il répare après coup, alors que la transaction empêche l'état intermédiaire d'exister. Un lecteur qui a lu pendant la fenêtre a déjà son chiffre faux.

© 2026 iAlexMG. Tous droits réservés.

Portabilité — Snowflake a un filet de plus, le voyage dans le temps du Cours 11 : il relit la table telle qu'elle était avant la requête fautive, tant qu'on reste dans la durée de rétention. Nommez alors la requête plutôt qu'un délai approximatif, pour revenir exactement à l'état d'avant. La limite reste la même : le voyage répare, il ne protège pas le lecteur de la fenêtre.

Partie 9 — Ce que le projet vous a montré

Théorie : `Theorie_12_SQL.md`, sections 5 et 6.

Ce qui a été mobilisé	D'où ça vient	Ce qu'on aurait perdu à s'en passer
Le grain et le piège du `LEFT JOIN`	Cours 01	un nombre de commandes deux fois trop grand, des produits sans catégorie disparus
L'axe complet	Cours 05	une moyenne quotidienne calculée sur les seuls jours actifs
Les cadres explicites, `RANK`, la part du total	Cours 04	un cumul faux dès qu'une date porte deux lignes
`ROLLUP` et `GROUPING`	Cours 06	des sous-totaux qu'on ne distingue pas d'une valeur inconnue
Les anti-jointures	Cours 07	des mesures publiées sans savoir si la donnée les mérite
L'as-of join sur la date de début	Cours 09	un rapport de marge vide, ou faux, à cause de 200 intervalles cassés
Le plan et la sargabilité	Cours 10	un rapport juste et inutilisable
La transaction et l'idempotence	Cours 11	un rapport qui affiche zéro une nuit sur trente

La dernière ligne est celle qui compte. Un lecteur pressé aurait pu sauter la partie B de la formation, et il s'en serait aperçu ici : c'est elle qui transforme un ensemble de requêtes justes en quelque chose qu'on peut confier à quelqu'un d'autre.

Nettoyage

```sql
DROP SCHEMA IF EXISTS RAPPORT CASCADE;
```


© 2026 iAlexMG. Tous droits réservés.

Gardez plutôt vos vues si vous voulez conserver le livrable : c'est le meilleur exemple de code que vous puissiez montrer en entretien, et il vaut mieux que n'importe quelle liste de mots-clés sur un curriculum.

Et ensuite ?

Vous avez terminé la formation. Ce que vous avez construit ici est la couche SQL d'un projet analytique complet : cadrée, contrôlée, mesurée, et rejouable.

Trois prolongements existent, et ils vivent ailleurs. L'orchestration de ce chargement, ses dépendances et ses tests automatiques sont le sujet de la formation Analytics Engineering, avec dbt. La restitution, son modèle sémantique et ses filtres appartiennent à la même formation, du côté de Power BI. L'administration de la plateforme, les rôles et les entrepôts de calcul sont le sujet de la formation Snowflake.

Une dernière étape avant de refermer : les exercices, dans `Exercices_corriges_12_SQL.md`. Ils ne demandent presque pas de SQL. Ils demandent de défendre ce que vous venez de faire, ce qui est exactement l'exercice d'un entretien technique.

© 2026 iAlexMG. Tous droits réservés.